

Formalizing Lieb's Concavity Theorem

E501C

Introduction

This essay revolves around my journey towards formalising a key theorem in Quantum Information Theory — *Lieb's Concavity Theorem* — in Lean 4. Our main result is now formalised:

Lieb's Concavity Theorem

[View Source](#)

```
223 variable {n : Type*} [Fintype n] [DecidableEq n]
224
225 /-- abbrev for n by n complex matrices -/
226 local notation "M[" n "]" => Matrix n n ℂ
227
228 /-- **Lieb's Concavity Theorem** for matrices [Nik2013, Thm 1.2(a)]:
229   `(A, B) ↦ re ((A ^ s * K * B ^ (1 - s) * K†).trace)` is jointly concave in `(A, B)`
230   for `0 < s < 1`, `A` positive definite and `B` positive semidefinite -/
231 theorem LiebConcavity_matrix {s : ℝ} (hs : 0 < s ∧ s < 1) (K : M[n]) :
232   ConcaveOn ℝ {(A, B) : M[n] × M[n] | A.PosDef ∧ B.PosSemidef}
233   (fun (A, B) => re ((A ^ s * K * B ^ (1 - s) * K†).trace)) := by
234   convert LiebConcavity hs K using 1
235   simp_rw [isStrictlyPositive_iff_posDef, nonneg_iff_posSemidef]
```

The source code documentations, project blueprints, and the GitHub repo are all publicly available in this website [ei25].

For me, the most non-trivial aspect of this project is not the theorems, nor even the background mathematics, but *how to allow the computer encodes our mathematics*. I would like to use the chance of this essay to document the features of Lean in relation to some behind-scene obstacles in formalisation. It is written to audiences who are not expert in Lean or other proof-assistant, to newcomers like me.

I will first introduce the basic notions about theorem proving in Lean, then devotes an entire section to some nuanced features of Lean. There will be running examples — including the entire final section — from relevant moments in the development of this project, where we will also explain the underlying design principles, decision-makings, and “debugging” processes. It is not the purpose for this essay to become a tutorial or a comprehensive reference for Lean, the narrative is mainly to serve the practices of this project. More information can be found in Lean Community’s articles [LeanComm], the *Theorem Proving in Lean 4* [LeanThm], and *The Lean Language Reference* [LeanRef].

It is painful to help computer understand mathematics. Nevertheless, the pain is necessary as to meet the requirement of absolute preciseness and the deconstruction of concepts and definitions into crystal clear logical layers, something informal mathematics has done better than other disciplines but not enough.

Contents

Introduction 1

1 A Quick Tour to Theorem-Proving in Lean 3

- 1.1 Philosophy: Proposition as Types 3
- 1.2 Examples: Proofs with Tactics 4
 - 1.2.1 Substitution 5
 - 1.2.2 Pattern-matching 6
- 1.3 More on Types 7
 - 1.3.1 Bedrocks: Dependent Type and Type Universe 7
 - 1.3.2 Building Blocks: Inductive Types 8

2 Sources of Convenience and Complication 9

- 2.1 On Equality 9
 - 2.1.1 Syntactic Equality 9
 - 2.1.2 Propositional and Definitional Equality 9
 - 2.1.3 Unification: What Defines Definitional Equality 11
 - 2.1.4 Recursor: Substitution as Induction on Equality 11
- 2.2 Type Classes : Hierarchy of Mathematical Structures 13
 - 2.2.1 Type Classes as Polymorphic Predicates 13
 - 2.2.2 Bundled Approach: Morphism Class 15
 - 2.2.3 Nuances in Bundled and Unbundled Approaches 16
- 2.3 Instance Search and Efficiency Concerns 16
 - 2.3.1 Scope Control 17
 - 2.3.2 Priority Control 18
- 2.4 Instance Unification: the Million-pound Diamond Problem 18
 - 2.4.1 Scope Control by `scoped` and `local` 19
 - 2.4.2 Transparency Control 20

3 Action: From Matrix to HStarAlgebra 22

- 3.1 Mathematics 22
- 3.2 Main Declarations and Results 24
 - 3.2.1 Algebra Homomorphism and some Properties 25
 - 3.2.2 Star-Algebra Homomorphisms and CFC/Rpow Commutativity 26
- 3.3 Obstacles 29
 - 3.3.1 Puzzle with `extends` and Diamond Digging 29
 - 3.3.2 Completing the Hypothesis 31
 - 3.3.3 Efficiency Concerns 31

Conclusion and Outlook 34

1 A Quick Tour to Theorem-Proving in Lean

Lean is a programming language founded upon type theory: everything is (a term of) some type. It is also a proof assistant where we can (1) write codes that represent propositions and (2) writes codes that represent proofs of these propositions.

As a proof-assistant, Lean also has higher-level supports for the convenient of human users, including the ability to define new abbreviation or assigning meanings to symbols that makes the whole expressions looks more like ordinary mathematics. The **elaborator** is the key component of Lean that does the work to unfold these human-read expressions and syntaxes into fully explicit ones. These fully explicit expressions will then be passed to Lean’s **kernel**, and all that remains for kernel is rudimentary checks of type-matches and the rules are followed.¹ Lean also has meta-level support like **tactics** — meta-programmes written by users to help auto-fill in the term-construction so as to remove redundancy and improve readability.

Finally, we have **Mathlib** — the de facto library of formalised mathematical statements in Lean. It is not complete, but sufficiently wide-ranging and general to be able to give foundation-level support for many downstream formalisation tasks.

This is how the whole system works. A powerful playground is ready to construct our mathematics upon. Before us playing, we need to understand how propositions and proofs work in Lean.

1.1 Philosophy: Proposition as Types

Propositions in Lean are also modelled using its type theory. **Prop** is the type whose terms are the individual propositions. The more interesting thing is how proofs are modelled: in Lean, each proposition $p : \mathbf{Prop}$ is itself a type, so it can carry terms; a term $h : p$ is interpreted as “a proof of p ”. A “false” proposition would be those with empty terms, i.e. no proofs. That’s it, no extra logical component or functionality is introduced to the language, it’s just all types and terms.

Modelling propositions as types and proofs as terms means their construction can only submit to the rules of the underlying type system. But we still have to raise the central doubt: can these type-formulating rules faithfully model logical rules so that constructing terms can be faithfully interpreted as the activity of constructing proofs? Let us start with a toy example of a “proof” in Lean:

```
variable (p : Prop)
example : p → p := fun (h : p) => (h : p)
```

We declared a proposition p in the environment. Using jargon, in the example we have constructed a term of $p \rightarrow p$ (a *function type*, one of the basic ways to construct new types) as the function that takes a term h of p to h itself. But note, if we say a term is a proof, we then can rephrase this as: a proof of p gives a proof of p — isn’t this “ p implies p ”? More generally, we can see that it makes sense to model “implication” by a “function type”: proving that p implies q (constructively) amounts to providing a function that associates any proof of p with a proof of q .

```
variable (p q : Prop)
example : p ∧ q → q ∧ p := fun (⟨h1, h2⟩ : p ∧ q) => (⟨h2, h1⟩ : q ∧ p)
```

¹That’s why kernel has very minimal code-base, less-prone to bugs (i.e. a wrong proof pass the check). Although it is still an open task to formally verify its bug-freeness, after years no such bug has been identified, people have faith on the maintainers and so many mathematical contents have been built atop it, so people insist to call it a “trusted” kernel. We rarely talk about whether we should trust the elaborator, because we don’t need to. Its bugs will be propagated to the kernel, i.e. the fully explicit expression is not well-typed, so kernel will reject it anyway.

This example ostentatiously shows how “ p and q ” is modelled in Lean: as a “product type”, whose term is a tuple with entries being terms (proofs) of p , q respectively. This also makes sense, a proof of “ p and q ” is just “a proof of p , and a proof of q ”. To prove the above implication, we construct the function that simply flip the two entries in the output tuple.

The interpretations of “and”, “implies” as “product type”, “function type” is a sublist of **Curry–Howard Isomorphism** [How80; SU06], which establishes an one-to-one correspondence between type-theoretical and logical concepts. This is the formal justification of the faithfulness of nearly all proof assistants nowadays. For the machine, “verify the correctness of proof” then comes down to “check the term is constructed correctly”: (1) only the legal type-construction rules are used, and (2) the resulting term’s pattern does match the intended type. In the above example, if we accidentally write `fun (⟨h1, h2⟩ : p ∧ q) => ⟨h1, h2⟩`, we end up construct a term of $p \wedge q \rightarrow p \wedge q$, type-mismatch error will fire (proved a different proposition!).

In the remaining part of this essay, in the context of theorem-proving, we will speak about “terms” and “proofs”, “types” and “proposition” interchangeably.

1.2 Examples: Proofs with Tactics

In mathematics the theorem to be proved may have numerous hypothesis and preassigned variables. In principle, we can patch all of them as a single proposition with nested implications, but the nested form is not visually appealing. Lean’s syntax allows us to write (revisit `LiebConcavity`):

```
theorem LiebConcavity_matrix {s : ℝ} (hs : 0 < s ∧ s < 1) (K : M[n]) :
  ConcaveOn ℝ {(A, B) : M[n] × M[n] | A.PosDef ∧ B.PosSemidef}
  (fun (A, B) => re ((A ^ s * K * B ^ (1 - s) * K†).trace)) :=
  convert LiebConcavity hs K using 1
  simp_rw [isStrictlyPositive_iff_posDef, nonneg_iff_posSemidef]
```

Here `(hs : 0 < s ∧ s < 1)` and `(K : M[n])` are explicit hypothesis/variable. `{s : ℝ}`, with the curly brace, means this is an **implicit** variable, i.e. if we were to call this theorem in other places we don’t need to fill in s — Lean’s elaborator will automatically, from contextual information like the `hs` hypothesis, infer what s should be. Here, instead of proving a nested implication via nested `fun {s : ℝ} => (fun hs => ...)`, using this syntax all we need is to construct, with all these pieces before the semicolon `:`, a term of `ConcaveOn...`

Expanding out the definition of `ConcaveOn` will give us a horribly looking nested proposition within which each component can be reduced as a further nested expression.² The ordinary mathematics that are formalised nowadays usually has layers of layers of constructions, so writing out raw proof terms all at once by hand, as in the previous section, becomes impractical here.

We must stand upon the shoulders of others. We could call pre-established lemmas in Lean just like saying “according to lemma... we have...”. We could also use tactics to make the construction more manageable. Instead of constructing the full term at once, Lean allows us to work interactively, and at each step the **infoview** panel (supported by most IDEs) displays the current proof state — the hypotheses available to use and the goal (the “remaining thing” to construct the full term), then uses tactics or raw methods to inhabit the goal. Proof in Lean becomes, in effect, a goal-chasing game with toolkit.

Let us illustrate. In the proof body (line 253-254) of `LiebConcavity_matrix`, we want to call a more general theorem called `LiebConcavity` here to help the proof; the mysterious strings `convert` and `simp_rw` are examples of tactics.

²Imagine: if we explicitly spell out what it means for a function to be concave on a set, we need to introduce scalars, which are real numbers, that sum to 1, the underlying set is convex etc. Further down, we need to unfold the definition of set, of convex set, etc.

0. After `by` keyword, Lean enters *tactic mode*, and infoview displays the initial goal:

```
⊢ ConcaveOn ℝ {(A, B) | A.PosDef ∧ B.PosSemidef} ...
```

1. The `convert` then tells Lean: let's apply the already-proven `LiebConcavity hs K`, which takes the form `ConcaveOn ℝ {(A, B) | IsStrictlyPositive A ∧ 0 ≤ B}...`, and outputs what remains to be proved. Everywhere except the inner curly bracket of this expression is virtually the same³ with the initial goal, so we only to prove that this differed part is actually equal. Indeed, the goal state becomes:

```
⊢ {(A, B) | A.PosDef ∧ B.PosSemidef} = {(A, B) | IsStrictlyPositive A ∧ 0 ≤ B}
```

2. The `simp_rw` calls the results in the brackets, which are lemmas that say, a matrix is strictly positive $\Leftrightarrow$ positive definite, etc., and substitute the expressions using these iffs. Goal accomplished.

There are actually more than one way to proceed with the proof using tactics, so there is a room of freedom. But underneath the hood these tactics are performing just the following basic actions: *substitution* and *application of known results*.

1.2.1 Substitution

Here, we assume for the present “=” in Lean behaves as one would expect : we may “move freely” from one side of the equality to another, replace expression at one side freely to the other one. This is big lie, but we defer the complication to section 2.1 for better content design.

Substitution is arguably the single most fundamental mechanism in proof construction — nearly every non-trivial proof involves rewriting some sub-expressions with an established equality. Our `simp_rw` is actually the child of two other most basic and widely used tactics:

- `rw [h]` (rewrites). It did what the name suggests: for $h : e_1 = e_2$, `rw[h]` finds the first occurrence of expression e_1 and replace it by e_2 . It is the foundation of all substitution tactics. Besides equality, `rw` also works on propositional “iff”.
- `simp` (simplification). The first “magic” tactic. It is backed up by a database of lemmas. While `rw` perform specific, one-time substitution, `simp` can automatically and repeatedly substitute using lemmas from its database and ones called in the bracket. It is basically impossible, though, to guess what it ends up with, so trial-and-errors are inevitable in practice. We are also allowed to add our own lemma to its database by tagging this lemma with `@[simp]`.

In many cases, a suitable `simp[...]` call will drastically simplify the expression and even close the goal, so it is very widely applied.

Example: Use of `simp`

 [View Source](#)

```
69 inner_mul_right {a x y} := by
70   simp [frobenius_inner_def, conjTranspose_mul, star_eq_conjTranspose, mul_assoc]
```

Above we are proving that Frobenius (trace) inner product satisfies: for all n by n complex matrices A, X, Y , $\langle XA, Y \rangle = \langle X, YA^* \rangle$, where the star operation in context is conjugate transpose. `simp` applies the relevant lemmas to both sides of the equation and close the goal in one shot.

There is also the `tactic_name only [...]` syntax, which tells the tactic to try only the lemmas called in the bracket.⁴ In practice, it may happened that `simp` alone could waste a few seconds trying combinations

³Here, more precisely, the *definitional equality* is played. See section 2.1.2.

⁴Hence, substitution tactics without database-support like `rw` don't fit with this syntax.

of wrong lemmas in the database; it may also happen that `simp` did too much that makes the resulting expression unmatched with what we intend to work on in next step. We switched frequently to `simp only` for the above reasons, for example when performing complicated computations with matrices as necessary explicit steps to prove Operator Jensen's Inequality.

1.2.2 Pattern-matching

Any kind of tactics that do lemma application is underpinned by *pattern-matching*: Lean compares the present goal with the theorem/hypothesis to be applied and outputs the missing piece to construct. Although these tactics don't help release proof burden as `simp` does (since one still need to explicit call result to be applied), they can help organise the code logic by focusing the proof task on the most critical part.

Here are tactics more commonly used than `convert`:

- `exact`: Directly construct the term that matches exactly with the goal, either by directly calling an existing result or combined with raw constructions. One of the most basic tactics.
- `apply`: Literally apply a known result (usually stronger than the to-be-proved), but also allows leaving the hypotheses unfilled: turning them to subsequent goals instead. It can be used as an alias for `exact`, though, if we directly fill in the hypotheses fields, but if some of the hypotheses need multiple steps, delaying their proofs makes a better code design. For example:

Example: Use of `apply`

[View Source](#)

```
619   _ ≤ ∑ k, (1 / (↑(n + 2) : ℝ)) • cfc f (star (v k) * X a * v k) := by
620     apply hconv.map_sum_le
621     · intro k _; positivity
622     · simp [Finset.sum_const]; field_simp
623     · exact fun k _ => hXa_conj k
```

The goal state at line 619 asks us to prove a special form of Jensen's inequality. We apply a more general version of Jensen `hconv.map_sum_le`,⁵ then in next three dotted-subsections filling out the proof of three of its hypotheses, including the scalars in the sum are nonnegative, the scalars add to one, and each `star (v k) * X a * v k` belongs to the underlying set.

- `refine`: This tactic exemplifies the pattern-matching mechanism. It can help to separate out the more complicated term-construction sub-steps from the term's "main structure". For example:

Example: Use of `refine`

[View Source](#)

```
566   refine fun k => ⟨?_, ?_⟩
567   · exact Diag_isSelfAdjoint ha |>.conjugate' u |>.conjugate' (v k)
568   · calc spectrum R (star (v k) * X a * v k)
569     = spectrum R (X a) :=
570       Unitary.spectrum_star_left_conjugate (U := ⟨v k, V_mem_unitary k⟩)
571   _ = spectrum R (Diag a) :=
572       Unitary.spectrum_star_left_conjugate (U := ⟨u, U_mem_unitary hb⟩)
573   _ ≤ I := Diag_spectrum_subset ha_spec
```

The goal state before using `refine` takes the form $\forall k, \text{Property}_1 k \wedge \text{Property}_2 k$, so its proof must take the form `fun k => ⟨?_, ?_⟩`.⁶ The two `?_` holes are to be unified with proofs of the two

⁵Strictly speaking it is the theorem `ConvexOn.map_sum_le` applied, but here, it is analogous mechanism of *methods calling* in OOP by using *dot notation*, where we can replace `ConvexOn` by a concrete hypothesis `hconv` of same type.

⁶It is unsurprising that universal quantification can be naturally modelled by function type.

propositions in k , but again are delayed to new goals.

There are other tactics of different natures. For example, speaking of organising workflow, using `have` tactic we can manually introduce “intermediate lemmas” if we want to separate out its proof for modularity or the result is repeated used in later steps. There are also tactics that solve particular mathematical problems backed up by a library of Mathlib lemmas or even external algorithms, for example linear (in)equality solver `linarith`, ring equation solver `ring`, comparing-with-zero automation `positivity`, etc. Only experiences can tell what tactics suit in a particular circumstance.

1.3 More on Types

We now move a little more detail into Lean’s type theory. This section is for background information only; they belong to the foundation of Lean’s type theory and mostly appear as the “linguistic” part of the formalisation task, hence we have intentionally omitted many details.

1.3.1 Bedrocks: Dependent Type and Type Universe

What distinguish Lean (and Coq) from other proof assistants based on type theory is that it allows **dependent type**, which is roughly “type with parameter”. This notion is necessary if we were to stick with the propositions-as-types paradigm. Consider the proposition with universal quantifier: $\forall x : \alpha, P\ x$; it says that for every $x : \alpha$, there is a proof of $P\ x$. A proof of this proposition is therefore a function that, given any $x : \alpha$, produces a term of type $P\ x$ — but here the output type $P\ x$ varies with the input x . This cannot be expressed as a simple function type $\alpha \rightarrow ?$, since the returned type is not fixed. The solution is the *dependent function type* (also called a *Pi type*), whose expression takes the form “ $(x : \alpha) \rightarrow P\ x$ ”. A term of this type is a “function” whose return type depends on the input.⁷

For our purpose, it suffices to know that dependent types simply makes Lean much more expressive, allowing us to write expressions like “`Ring  $\alpha$` ” (which says “ α is a ring”)⁸ with α replaceable by any concrete type, e.g. “`Ring  $\mathbb{R}$` ” (the real numbers form a ring).

There are also some special types usually overlooked as “implicit from context”. The key idea is that every type in Lean must itself be of some type: the type of `Nat` is `Type`, the type of `Type` is `Type 1`, the type of `Type 1` is `Type 2`, and so on. This hierarchical design avoids the type-theoretic analogue of Russell’s paradox that “a universal type being a member of itself”. These `Type u` are called **type universes** and u is a **universe level** ranging from 0 to infinity; in particular, `Type 0` represents `Type`.⁹

In practice, most Mathlib declarations are *universe-polymorphic*: rather than fixing a specific universe level, they work for any u . The idiomatic way to declare a variable ranging over an arbitrary type is “`A : Type*`”, which is a shorthand for introducing a universe variable u and writing `A : Type u`. For our purpose, it suffices to know that whenever we write `{A : Type*} ...`, we intend to say “let A be ...”

⁷In fact, simple function types $\alpha \rightarrow \beta$ are the special case where $P\ x = \beta$ is constant. There is another dependent type called *Sigma type* that also models product type and the existential quantification. We won’t explain the detail, the quick take-away is that: the *these dependent types subsume all simple types*. Hence, Lean only needs to have them as its foundation. More details can be found in the chapter “Dependent Type Theory” of *Theorem Proving in Lean 4*.

⁸This is an example of *type class*, see section 2.2

⁹Lean uses a more unified notation, “`Sort u`”: `Type u` is represented by `Sort (u+1)`, and the left-over `Sort 0` is used to represent `Prop`. The cleverness of this design lies in the clean implementation of universe level expressions that consume *impredicativity* of propositions, which is the powerful feature that: we can form propositions that quantify over propositions and all other types, but the result still stays in `Prop` rather than in some higher type universe level. Readers may consult the section “Level Expressions” of *The Lean Language Reference*.

1.3.2 Building Blocks: Inductive Types

Inductive type is the primary mechanism to define a new type in Lean. It constructs new type by listing the admissible ways to construct its terms. We call these ways as **constructors**, or *introduction rules* for logical types. The key is that these listed constructors are exhaustive — every term of the type must arise from one of them.

Logical connectives are defined using inductive types. For example, “Or” has two constructors:

```
inductive Or (a b : Prop) : Prop where
| inl (h : a) : Or a b
| inr (h : b) : Or a b
```

This definition reflects the two ways to prove a disjunction: to prove $a \vee b$, one must either provide a proof of a (via `Or.inl`) or a proof of b (via `Or.inr`). Hence, for example, in Lean the proposition $a \rightarrow \text{Or } a \ b$ can be proved by `fun (h : a) => Or.inl h`.

A more interesting example is the type of natural numbers `Nat`. There are infinitely many natural numbers, but we cannot list them one by one since we are only allowed to write finite lines of codes. `Nat` thus follows Peano that defines natural numbers in terms of zero and a successor operation:

```
inductive Nat : Type where
| zero : Nat
| succ (n : Nat) : Nat
```

The constructor `Nat.succ` cleverly exploits the use of recursive call; accordingly, every natural number is either zero or as a successor of a another one, so all admissible terms of `Nat` are of the form “some finite number of `Nat.succ` composed and applies to `Nat.zero`”, in bijection with natural numbers.

Finally, consider the following example of points in a grid indexed by natural numbers:

```
structure Point : Type where
  x : Nat
  y : Nat
```

This is the syntactic sugar for this inductive type:

```
inductive Point : Type where
| mk (x : Nat) (y : Nat) : Point
```

The syntax `structure` allows the inductive type with one constructor but multiple parameters to be declared like a data structure with multiple fields. It, and its variation `class`, are commonly used to build mathematical structures that are essentially “something with a list of properties”, which we will discuss in section 2.2.

2 Sources of Convenience and Complication

Now we get deeper on some special features of Lean that show up to be not only relevant to practices but also has interesting questions to ask of themselves. We will discuss nuances with equality in Lean, then one of the most important set of declarations in Lean — “type classes”.

2.1 On Equality

In ordinary mathematics aside from foundations, equality is a single, unambiguous notion. In Lean, there are (at least) three different notions of equalities, and understanding their distinction is essential to understanding why certain proof work and why others frustratingly do not. For more info on equalities, see “Equality” of *Theorem Proving in Lean*.

2.1.1 Syntactic Equality

Two expressions are **syntactically equal** if they are, literally the same string of symbols. This is the strongest and most rigid notion. It is the level at which `rw` and `simp` operate when it searches for matching expression.¹⁰

Actually not quite. Lean does slightly smarter work than just comparing raw strings:

- **Variable renaming.** Lean sees $\forall x \exists y, P x y$ and $\forall t \exists x, P t x$ to be syntactically equal. In other words, Lean does not care what precise symbol we attach to the variables. This is underpinned by the formal rule “ α -equivalence” which originates in Lambda Calculus. It really is a foundational feature rather than an exception, not only for Lean but also any system that uses variable binding.
- **Notation.** Declaring `notation "M[" n "]" => Matrix n n C`, Lean sees `M[n]` and `Matrix n n C` as syntactically equal expressions. Making notation only creates *macro* for the parser, i.e., think of the notation and the expression it represents are *keys of the same value* in some dictionary, so after they are parsed Lean reads them the same.

Aside from the above two “exceptions”, there is nothing significant about syntactical equality.

2.1.2 Propositional and Definitional Equality

The other two notions are much more subtle, but let us be informal and hand-wavy first to get the taste.

- Two expressions a and b are **propositionally equal** if there is a *proof* $h : a = b$ that they are equal. This is mathematicians’ familiar notion of equality.
- Two expressions a and b are **definitionally equality** if the elaborator can automatically discover by means of unfolding definitions and reducing them to same expression.

For the present moment, it suffices to take definitionally equal means, literally, “equal by definition”. One may say definitional equality is a special kind of propositional equality, because we can also prove it — effortlessly though, using the “tactic” `rfl`.

```
example: 2 + 2 = 4 := rfl
```

¹⁰Strictly speaking, they can do a little more than looking for mere string matches — this involves the notion of “reducibility”, see Section 2.4.2.

Here `rfl` performs a magic. Just to be sure, `rfl` is not a calculator. This proof works simply because the elaborator can unfold the definition of 2, 4, and addition for natural numbers, turning $2 + 2$ and 4 to a same canonical expression.¹¹ `rfl` confirms that $2 + 2$ and 4 are equal by definition.

Here is an infamous example that illustrates a non-definitional equality: for general natural number n , $n = n + 0$ can be proved by `rfl`, but $n = 0 + n$ cannot! The reason is that addition for natural number is so defined so that the elaborator can automatically reduce $n + 0$ but it gets stuck on $0 + n$.

Example: Definition Unfold

[View Source](#)

```
37 @[simp]
38 theorem frobenius_inner_def (A B : Matrix n n k) :
39   ⟨A, B⟩_k = (B * A†).trace := rfl
```

This lemma is a single definition unfolding.¹² The inner product in context is defined to be right hand side using trace, hence `rfl` closes the proof. This result looks redundant, for it did nothing but restating the definition. However, it is a common practice in Mathlib to write a lemma like this and tagged with `@[simp]`, immediately after newly defined concept (usually important or complex ones). Here is the reason. Recall that when searching for matching strings, `rw` and `simp` operate at the level of syntactical equality. They cannot pierce through the definition when trying to rewrite:

```
def a := b
--- infoview displays: `simp` made no progress
example (h : b = c) : c = a := by simp [h]
```

With `@[simp]` tagged to the associated “useless” lemma, `simp` now practically acts like it is able to automatically “unfold” definition, `simp[h]` above will work.

Tactics like `exact`, `apply`, `refine`, and `convert` all work up to definitional equality. Let us revisit the old example 1.3 on `refine`. In Lean, the membership predicate with set-builder $y \in \{x : A \mid P\ x\}$ is definitionally equal to $P\ y$. This is why in the `refine` example, while the goal actually take the form `star (v k) * X a * v k ∈ {a | IsSelfAdjoint a ∧ spectrum ℝ a ⊆ I}`, it could be directly matched with `<?, ?>` pattern that represents a conjunction.

There are also tactics that do nothing more than rewriting with definitions. For example:

Example: Definition Rewrites by `dsimp`

[View Source](#)

```
193 let F : A → A := cfc f
194 let G : A → A := cfc g
195 let R := a • R1 + b • R2
196 ...
197 (GenPerspective A f g) (a • L1 + b • L2, a • R1 + b • R2)
198 = G R ^ ½ * F (G R ^ (-½)) * (a • L1 + b • L2) * G R ^ (-½)) * G R ^ ½ := by
199   dsimp only [GenPerspective]
```

In this proof step we just expand the `GenPerspective` plus automatically inserting local definitions `F`, `G`, `R` in order to match the goal with particular expression more convenient to simplify. `simp` is less smart here because it requires us to manually type `F`, `G`, `R` into bracket.

What makes them able to pierce through definitional equality? The source of such power is not in the tactics themselves, it lies in Lean’s **unification** mechanism. We now explain.

¹¹`Nat.succ (Nat.succ (Nat.succ (Nat.succ Nat.zero)))`.

¹²To Lean `theorem` and `lemma` declarations make no difference. It’s just different names.

2.1.3 Unification: What Defines Definitional Equality

Propositional equality is not a primitive logical component in Lean. It is an exotic inductive type:

The Equality Relation

[View Source](#)

```
73 inductive Eq :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$  where
74   | refl (a :  $\alpha$ ) : Eq a a
```

The expression “ $a = b$ ” is a notation for “ $\text{Eq } a \ b$ ”. What this definition seemingly suggest is that the only kind of equality we can prove in Lean is the trivial syntactic equality $\text{Eq } a \ a$ via the introduction rule $\text{Eq.refl } a$. However, this introduction rule is more powerful than it appear.

Revisit the equality $\text{Eq } (2 + 2) \ 4$. This, magically, can be proved by $:= \text{Eq.refl } 4$. Just from the surface the type of $\text{Eq.refl } 4$ should be $\text{Eq } 4 \ 4$ which looks unmatched with the goal $\text{Eq } (2+2) \ 4$. Crucially, Lean’s intelligence does not stop here. The elaborator then tries to *unify* these two types, and in unification it *will try* to reduce both sides and check for a potential match. Thanks for this effort, both $\text{Eq } 4 \ 4$ and $\text{Eq } (2 + 2) \ 4$ are reduced to the same expression $\text{Eq } (\text{Nat.succ } (...)) (\text{Nat.succ } (...))$, matched.

It is for the same reason that those pattern-matching tactics can detect definitional equality: when the input and target types present, none of these tactics per se checked their definitional equality, they all trigger the meticulous elaborator to perform the check at the unification.¹³ `refl` is in fact an abbreviation for $\text{Eq.refl } _$ (so strictly speaking not a tactic¹⁴), that’s why it can be used to prove definitional equality.

In essence, the proof we provide for other equalities that cannot be directly closed by `refl` are “hints” or “road-map” for the elaborator to navigate user-defined structures and do more complicated reductions. What this implies, furthermore, is that all provable equalities are, in principle, equal by definitions.¹⁵ Nevertheless, we want to highlight the boundary between “what proofs can be done by elaborator and what cannot”, so we capture it with the terminology “definitional equality”. The importance of such boundary will become more evident when we discuss the “diamond issue” in section 2.4.

2.1.4 Recursor: Substitution as Induction on Equality

Finally, we talk about the underlying mechanism of substitution.

From the logical point of view, substitution can be formulated as **Leibneiz’ Law**: if some property (motive) holds for a and $a = b$, then it holds for b . This is declared as a theorem scoped in `Eq`:

Substitution as a Theorem in Lean 4

[View Source](#)

```
369 theorem Eq.subst { $\alpha$  : Sort u} {motive :  $\alpha \rightarrow \text{Prop}$ } {a b :  $\alpha$ }
370   (h1 : a = b) (h2 : motive a) : motive b :=
371   Eq.ndrec h2 h1
```

From its proof we can see the secret of substitution lies in the mysterious “`Eq.ndrec`”. This is an (non-dependent) abbreviation for “`Eq.rec`”. Let’s look at what it is.

Every inductive type comes with a *recursor* `Some_inductive_type.rec`. This is auto-generated by Lean’s in-built API for inductive type. To illustrate, let us compare the recursors for `Nat`, `Eq` and `Or`:

¹³More precisely, it is the “`isDefEq`” command being triggered.

¹⁴Although there is a tactic also called `refl`, used in usual tactic mode after `by`, which is a meta-programme more powerful than `Eq.refl`.

¹⁵Modulo in-built axioms in Lean, for example **Function Extensionality** (`funext`) which says two functions $f \ g : \alpha \rightarrow \beta$ are equal if and only if $\forall x : \alpha, f \ x = g \ x$. But of course set aside the philosophical nudges one could say this *is* the definition of functions being equal.

```

#check Nat.rec
⊢ {motive : N → Sort u}
  → motive Nat.zero → ((n : Nat) → motive n → motive n.succ)
  → (t : Nat) → motive t

#check Eq.rec
⊢ {motive : (b : α) → a = b → Sort u}
  → motive a (Eq.refl a)
  → {b : α} → (t : Eq a b) → motive b t

#check Or.rec
⊢ {motive : a ∨ b → Prop}
  → ((h : a) → motive (Or.inl h)) → ((h : b) → motive (Or.inr h))
  → (t : a ∨ b) → motive t

```

Recursors in these explicit forms look confusing, but actually they are trying to represent some really foundational concepts. For convenient we can read “motive” as “property” (recall that `Sort 0` is `Prop`). Then:

- `Nat.rec` says: for any property on natural numbers:

If it holds for 0, and whenever it holds for n implies it holds for its successor, then it holds for any $t : \text{Nat}$. This is the **Induction Principle**!

- `Eq.rec` says: given any property for terms of α (and for proofs of equality with a):

If a (and the proof `Eq.refl a`) satisfies it, then b (and any proof $t : a = b$) satisfies it.

This still reads like nonsense, especially with the dependency on (for proof of ...) explicated. However, we can remove such dependency as follows: given a property $P : \alpha \rightarrow \text{Prop}$, define the motive

```
motive : (b : α) → a = b → Prop := fun (b : α) t => P b
```

This motive is essentially non-dependent on proof of $a = b$.¹⁶ Then the recursor reads: if a satisfies P , then b with $a = b$ satisfies P . This is precisely **Leibniz Law**!

- `Or.rec` says: for any property (on proof of $a \vee b$):

If, when a has a proof, it holds (on proof of $a \vee b$ via left introduction), and if when b has a proof it holds (on proof of $a \vee b$ via right introduction), then it holds (given any proof of $a \vee b$). Like `Eq.ndrec`, there is an `Or.elim` that removes the dependency on proof of $a \vee b$ (read with these brackets removed), leaving us **Or Elimination Rule**!

Recursor represents a sort of general induction principle: to verify that some property holds for all terms of this inductive type, it suffices to verify it on its term constructors. The interpretation is fairly intuitive for data-carrier inductive types like `Nat`, but when interpret on propositional-valued inductive types like `Eq`, `Or` and remove proof-dependency, the logical rules are miraculously recovered. There is probably no better explanation for this phenomenon other than reassuring the readers that: recursor is yet another manifestation of Curry-Howard Correspondence. Logic rules do not need a separate layer to define, they are manifested as the inevitable ways we interact with the data structures that represent the logical components.

`induction` is a tactic that built atop of the facility of recursor. Its applicability is not limited to propositions about natural numbers. We will showcase one example in section 3.2.1.

¹⁶Actually, we have just partially reconstructed the definition of `Eq.ndrec`.

2.2 Type Classes : Hierarchy of Mathematical Structures

Aside from proof, there is another key aspect of modern mathematics: making definitions. If we are envisioning the more ambiguous goal of formalising an entire mathematical theory in a single library, like Mathlib, we need to carefully design how the definitions are made so that not only allows Lean to correctly and more efficiently retrieve and manipulate these “knowledge”, but also ease maintainers’ work of organising and updating the code space.

Suppose Lean now has a definition for “inner product space”. We want Lean be able to automatically know all the relevant sub-structures implied by being an inner product space, e.g. carrying real/complex-valued bilinear operation, a normed-space, ... all the way down to some basic information like “having an element called zero”, “having a binary operation called addition”, etc. For complicated structure like this, it soon become intractable to have all these predicates explicitly and simultaneously store under the declaration. Why not, just as in ordinary mathematics, define inner product space to be “linear space + a real/complex valued binary operation + properties of this operation”, and let Lean infer the more fine-grained sub-structures from these components. This is the basic idea of *type class system*.

2.2.1 Type Classes as Polymorphic Predicates

Almost all mathematical statements start with “Let X have some properties.” This is a hypothesis with a strong discriminative attitude: we do not care about what X -in-itself is — let it be “any set” if we have set theoretical foundation or “any type” in Lean with type-theoretical foundation — we only care about the properties which contain the actual mathematical content. X is here only because properties need a specific object to inhabit on. In Lean, we use **type classes** to assign predicates or data assign to polymorphic types.

For example, to say “let α be a ring”, we would write:

```
variable {α : Type*} [Ring α]
```

This `Ring` living inside the square bracket is an example of type class. The effect of which is that, now some extra data and lemmas are added into the environment, so that α now behave like an ordinary ring. Another example from Lieb’s Concavity Theorem declaration:

```
variable {n : Type*} [Fintype n] [DecidableEq n]
```

The `Fintype` and `DecidableEq` type classes roughly represent the predicates that “ n is like a finite set” and “any two terms of n are either propositionally equal or not”.¹⁷ One can thus read this line as “Let n be with only finitely many terms.” This `[Fintype n] [DecidableEq n]` combination is a common abstraction level for (decidable) theorems about finite-dimensional matrices in Mathlib.

Each of these type classes is themselves a type. To declare a new type class, we use the `class` syntax.

From Addition to Additive Semigroup

[View Source](#)

```
178 class Add (α : Type u) where
179   /-- `add a b` or `a + b` computes the sum of `a` and `b`. -/
180   add : α → α → α
181
182 /-- An additive semigroup is a type with an associative `(+)`. -/
183 class AddSemigroup (G : Type u) extends Add G where
```

Continued on next page...

¹⁷which assumes law of exclude middle. This law belongs to classical logic which has no simple type-theoretic counterpart, hence it needs to be treated as a separate “classical logic” axiom.

```

184  /-- Addition is associative -/
185  protected add_assoc : ∀ a b c : G, a + b + c = a + (b + c)

```

The sole thing of declaring `[Add α]` is that now α has an addition operation — we know nothing about it other than it is a binary operation. Then, using the `extends` syntax, the type class `AddSemigroup` *inherits* the class `Add` plus an extra axiom `add_assoc`. After declaring `[AddSemigroup α]`, α now has a binary operation “add / +” that satisfies the associativity.

So a type class is like a data structure carrying some term fields. Each field can contain either a concrete data — like a function `add` — or a proposition — like `add_assoc` — that together form a synthetic predicate. The `extends` syntax allows one to bring the data fields of already-defined classes together to form a more complicated class; one can readily envision that such an activity enables us to build a hierarchical graph of mathematical structures with each type class being a node and arrows denoting the `extends` relation. Readers are invited to see [Mathlib20, Figure 1] which is a partial structural hierarchy of type classes that build up “normed field”.¹⁸

Now we can define and store these class definitions in a structural way, but we want to achieve more. We want Lean to be able to, for example, from `[Ring α]` infer that α must have a commutative addition operation. In Lean, such an intelligence is achieved by the mechanism of *instance inference*.

An *instance* of a type class is just a term of which, i.e. a witness / example of the predicate / structure represented by the class. In fact, the real effect of writing `[SomeClass α]` is to declare a hypothetical instance `h : SomeClass α` in the environment. There are two ways one can build an instance.

- (a) Lean has silently generated some straightforward instances for us when using the `extends` syntax. For example, having `Ring (R : Type*) extends Semiring R, AddCommGroup R, AddGroupWithOne R`, we automatically have instances `Ring.toSemiring : Semiring R`, `Ring.toAddCommGroup : AddCommGroup R` and `Ring.toAddGroupWithOne : AddGroupWithOne R`.
- (b) We humans manually instantiate. For example, in our project we declared an instance of inner product space structure on n by n matrices using the Frobenius inner product:

Frobenius Inner Product Instance
[View Source](#)

```

55  /-- The Frobenius inner product on `Matrix n n k`. -/
56  @[instance_reducible, local instance]
57  def frobeniusInner : Inner k (Matrix n n k) where
58    inner A B := (B * A†).trace
59
60  @[instance_reducible, local instance]
61  def frobeniusInnerProductSpace : InnerProductSpace k (Matrix n n k) where
62    norm_sq_eq_re_inner := frobenius_norm_sq_eq_re_inner
63    conj_inner_symm := by
64      simp [starRingEnd_apply, ← trace_conjTranspose, conjTranspose_mul]
65    add_left := by simp [mul_add]
66    smul_left := by simp_rw [starRingEnd_apply]; simp

```

Here, we declared an instance for type class `Inner` specialised to `Matrix n n k`, for which we just need to provide a binary operation on `Matrix n n k` whose range is in k . Then, we get an instance of `InnerProductSpace` specialised to `Matrix n n k` by filling out the necessary proof fields, each corresponds to a property of the inner product function.

¹⁸Just to note: this figure is illustrative only. We are now using Lean 4 and Mathlib 4, but it comes from the age of Lean 3 and Mathlib 3 so the declarations have experienced significant changes.

All these auto-generated or human-defined instances will be accessible by Lean, serving as working knowledge implicit from the screen. Attentive readers may notice that no where in the definition of `frobeniusInnerProductSpace` have we mentioned the inner product instance `frobeniusInner`. Indeed, a fuller proof should also include this field: `inner := frobeniusInner`. But Lean's elaborator is powerful: it discovers the `frobeniusInner` instance from the file so it can auto-fill the `inner` field with it.

2.2.2 Bundled Approach: Morphism Class

We remark that `class` is in fact a special syntax built up from `structure` syntax. We could use `structure` plus some extra tags to achieve the similar effect of `class`, but this syntax is reserved for a special kind of type class that are called to be *bundled*.

This bundled approach is usually used in building the classes of algebraic homomorphism in Mathlib. We will compare them with the previous *unbundled* ones at the end. Now we begin with an elementary example of building the additive monoid homomorphism from scratch:

Structure Declaration of Additive Monoid Homomorphism

[View Source](#)

```

85 structure ZeroHom (M : Type*) (N : Type*) [Zero M] [Zero N] where
86   /-- The underlying function -/
87   protected toFun : M → N
88   /-- The proposition that the function preserves 0 -/
89   protected map_zero' : toFun 0 = 0
90
91 structure AddHom (M : Type*) (N : Type*) [Add M] [Add N] where
92   /-- The underlying function -/
93   protected toFun : M → N
94   /-- The proposition that the function preserves addition -/
95   protected map_add' : ∀ x y, toFun (x + y) = toFun x + toFun y
96
97 structure AddMonoidHom (M : Type*) (N : Type*) [AddZero M] [AddZero N]
98   extends ZeroHom M N, AddHom M N

```

They define, respectively, the zero-preserving, additive-preserving and additive-monoid homomorphism. Via `extends` syntax, `AddMonoidHom` inherits both structures, so that any `F : AddMonoidHom M N` has an underlying function `F.toFun : M → N` satisfies both `map_zero'` and `map_add'`. Also, with `extends`, Lean automatically generates two additional fields: `.toAddHom` and `.toZeroHom`, i.e. “forgetful functors” that given any `F : AddMonoidHom M N` outputs its additive-only version `F.toAddHom` or zero-only version `F.toZeroHom`.

In practice, one often want to construct some specific morphisms that are used to apply on elements. For example, in our project we bundles the `diagonal` map a star-algebra homomorphism, which is an infrastructure that supports continuous functional calculus deduction in diagonal matrix context:

Diagonal Map as a Star-Algebra Homomorphism

[View Source](#)

```

69 /-- `Matrix.diagonal` as a `StarAlgHom`. -/
70 def diagonalStarAlgHom : StarAlgHom R (n → α) (Matrix n n α) :=
71   { diagonalAlgHom R with
72     map_star' := fun v => by simp [star_eq_conjTranspose, diagonal_conjTranspose] }

```

`diagonal` is a function that sends an n -tuple $d : n \rightarrow \alpha$ to a diagonal matrix $: Matrix\ n\ n\ \alpha$ having d as its diagonal. In Mathlib, it was originally only bundled as an algebra homomorphism as `diagonalAlgHom`, we then build atop of it, fill in the proof field of `map_star'` (star-preserving) so that now we have all necessary data to get a star-algebra homomorphism whose underlying function is `diagonal`.

Mathlib also has various notations for these morphisms with which the hierarchical structure are made visually explicit. Examples that came across our project: ring homs $\alpha \rightarrow^{++} \beta$, star ring homs $\alpha \rightarrow^{*++} \beta$, R -algebra homs $\alpha \rightarrow_a [R] \beta$, star R -algebra homs $\alpha \rightarrow^*_a [R] \beta$, and continuous R -linear maps $\alpha \rightarrow_L [R] \beta$.

2.2.3 Nuances in Bundled and Unbundled Approaches

There are key differences in how the bundled classes and unbundled ones are used, all has to do with mathematical considerations.

- One consideration is that, for structures like ring, group, in a particular context there is only *one canonical structure* for each to consider. So for unbundled class like `Ring  $\alpha$` we allow only one canonical term (instance). But for homomorphism classes, we often need to deal with multiple terms.
- Related to the consideration of using multiple terms: we often want add, compose multiple ring homomorphism together, but it makes no sense to do such things for different ring structures. In other words, bundled classes themselves often also carry structures — algebraic, topological or order-theoretic — in contrast with the class `Ring  $\alpha$` of all possible ring structures on α that we view as structureless itself.

We summarise their different user cases in this table:

Feature	Bundled (e.g., <code>RingHom</code>)	Unbundled Class (e.g., <code>Ring</code>)
Declaration	<code>structure</code>	<code>class</code>
Uniqueness	Often multiple terms (maps).	One “canonical” ring structure.
Argument Usage	Explicit (<code>f : $\alpha \rightarrow^{++} \beta$</code>)	Implicit (<code>[Ring α]</code>)
Automation	Need to manually type the name.	Automatically inferred by elaborator.

Table 1: Comparison between Bundled Morphisms and Type Class Structures in Mathlib4

Remark 2.1. It is my opinion that even this table can be misleading. Bundled class really behaves and is used like ordinary types. More crucially, it does not enjoy the automation of the instance inference mechanism: we cannot write `[ $\alpha \rightarrow^{++} \beta$ ]`, and if we were to get a lower-level homomorphism from `f :  $\alpha \rightarrow^{++} \beta$` we need to manually provide the `.toSomeMap` field. Hence, besides the similarity in syntactical features like inheritance via `extends` and auto-generation of `.toSubStruct` fields, the bundled classes are heterogeneous, unparalleled with the unbundled ones. In fact, in many informal context when people talk about type class, they are thinking about instance inference, so they are actually thinking about these unbundled ones, these that live in the square brackets which act like predicates / environmental variables.

Hence, in the remaining part of this essay, *I will only call the unbundled ones as type classes*, and I will use descriptions like “bundled morphisms/approaches” to address the bundled ones.

2.3 Instance Search and Efficiency Concerns

When looking for needed instances, Lean can do *transitive search*: say starting with `[Ring  $R$ ]`, Lean not only can infer its “child”, for example `Semiring  $R$` via `Ring.toSemiring`, but also the “grand-child”, e.g. going further down the immediate instances to `Ring.toSemiring.toNonUnitalSemiring`. That’s all it can, any other instances we need to manually add to feed into the search graph.

Here is one case to consider: Lean cannot automatically see all theoretically possible sub-class of a given structure. For example:

```

374 -- A (unital, associative) ring is a not-necessarily-unital ring
375 instance Ring.toNonUnitalRing : NonUnitalRing α :=
376 { <Ring α> with }

```

Here, `instance` is another way to declare an instance. `NonUnitalRing` is not a grand-child of `Ring`, but its term field is a subset of which, and using `{ <Ring α> with }` the elaborator can fill it; now `NonUnitalRing` is an “adopted child” of `Ring`. Many theorems do not assume to have a unit in the ring but are often applied to common rings which do have units, so such an instance is necessary.

Besides the above instance for type class with generic parameter, we also want instances for specific objects that are “canonical”. For example the `Mul` instance for matrices defined as the usual matrix multiplication, the `MetricSpace` instance for real numbers induced by the usual absolute value, the `Ring` instance for the space of endomorphisms with multiplication being composition, etc. These need separate declarations too, and they are actually a significant part of the Mathlib’s content. Everything else (the grand-(adopted)child) is in principle discoverable by transitive search.

Efficiency also matters. We want Lean’s type class inference system to be able to start off from just a few declarations (mathematically compact assumptions) while also able to quickly find from them the individual instances (knowledges) required by the definition/theorem statements and the proof field. The two key constraints for performance faced by any computer programme are: space and time. To save both space and time to search, we need to (1) control the number of instances exposed to the elaborator, and (2) have an efficient search algorithm.

2.3.1 Scope Control

When elaborator starts to search, we don’t want to load its memory with too many instances, especially unnecessary ones so that it won’t waste time looking at these irrelevant paths. In order to be able to activate and deactivate the instances, we can use the technique of *scope control* via `namespace` declaration and the keyword `scoped`. We will discuss in section 2.4.1 a more crucial role that scope control plays.

For example, our formalisation of Lieb’s Concavity Theorem involves first a general, matrix-less version `LiebConcavity`, then we specialises to `LiebConcavity_matrix`. We surely need to import the file that contain relevant instances for matrices to be used in the specialised step, but they are not needed in the general context:

- On the side of the source file that contains relevant instances for matrices, notably the afore-mentioned `frobeniusInnerProductSpace` instance and “HStarAlgebra” instance, we put them under a namespace:

The FrobeniusMat Namespace

```

25 namespace FrobeniusMat
26 ...
27
28 /-- `Matrix n n ℓ` with the Frobenius inner product is an H*-algebra. -/
29 @[implicit_reducible, scoped instance]
30 noncomputable def instHStarAlgebra : HStarAlgebra ℓ (Matrix n n ℓ) where
31 ...

```

Inside `namespace FrobeniusMat`, every declaration automatically obtain prefix `FrobeniusMat.` to their name. If we want to explicitly use some declarations `dec_name` outside the namespace `FrobeniusMat`, then we need to either type the full name `FrobeniusMat.dec_name`, or type `open Frobenius` to avoid typing the prefix. But for implicit use of instances, the `scoped instance` tag creates the extra effect

that: if we are outside the namespace, then the elaborator can only see these scoped instances if we typed `open FrobeniusMat` or `open scoped FrobeniusMat`.¹⁹ This allows us to dynamically control the “visibility” of these instances.

- Now, at the file which proves Lieb’s Theorem:

Instance Scope Control in Theorem Proving
 [View Source](#)

```

11 public import LeanLiebConcavity.ForMathlib.Frobenius.Matrix
12 ...
13 theorem LiebConcavity {s : ℝ} (hs : 0 < s ∧ s < 1) (x : H) :
14   ConcaveOn ℝ {(A, B) : H × H | IsStrictlyPositive A ∧ 0 ≤ B}
15   (fun (A, B) => re ⟨x, A ^ s * x * B ^ (1 - s)⟩_C)
16 ...
17 --- open the packed instances on Matrix n n ℂ
18 open scoped FrobeniusMat
19 ...
20 theorem LiebConcavity_matrix {s : ℝ} (hs : 0 < s ∧ s < 1) (K : M[n]) :
21   ConcaveOn ℝ {(A, B) : M[n] × M[n] | A.PosDef ∧ B.PosSemidef}
22   (fun (A, B) => re ((A ^ s * K * B ^ (1 - s) * K†).trace))
23 ...
```

The first line imports the source file containing the above instances. Then those instances only become visible after the line `open scoped FrobeniusMat` and can be used and unified into `LiebConcavity_matrix` by the elaborator, without mathematicians explicitly dictating “let us now specialise to the Frobenius inner product...”.

2.3.2 Priority Control

Lean uses *depth-first* (with back-tracking) in instance search because *breadth-first* is too memory-heavy. However, the trade-off is that we have to deal with the possibility of the search path being too long or not going to work that cripple time-efficiency. One way around in Lean is to set “priority” parameter to an instance declaration, so that instances with higher priority gets searched first. We can either use fine-grained numerical values ranging from 0 to infinity, or the standard named values “low (= 100), mid (= 500), default (= 1000), high (= 10000)”.

One rule of thumb is “specific over general”. For example, consider inferring the commutative ring `CommRing` instance from the reals $\mathbb{R}$. One possibility is to use the generic instance `Field.toCommRing` (“every field is a commutative ring”) and then try to synthesis `Field ℝ`. But clearly, a better option would be to directly use the specific `CommRing ℝ` instance, if there is any (there is: `Real.commRing`). Generic instances as such apply to almost every search, so if Lean tries these first, it might waste time exploring a massive search tree; even worse, this extensive search might fail if the targeted object is not even a field. Hence, we can see everywhere in Mathlib some generic instances got priority 100 or even 50.

2.4 Instance Unification: the Million-pound Diamond Problem

An absolutely crucial feature of instance synthesis, that we have only briefly mentioned, is *Instance Subsingleton*, or the slogan “instances must be unique”. More precisely, this means for any given type, the type class inference system should only ever be able to find one canonical instance, or if multiple ones are found then they must be *definitionally equal*. This feature is necessary, because otherwise Lean will go insane figuring out, for example, what “ $a + b$ ” means if there are two virtually different `Add` present in the context.

¹⁹We remark that “`open`” makes every names within the namespaces visible, while `open scoped` only makes `scoped` names visible. Typically, instances are tagged with `scoped` keyword, while theorems and notations are not.

Remark 2.2. Since it is at the stage of elaboration the automated instance synthesis is happening, only definitional equalities can pass the unification. Hence, even if two instances are proved somewhere else to be equal, the elaborator won't know.

When the system discovers multiple paths to synthesise the instance, for example `Real.commRing` and `Real.instField.toCommRing` being two paths to arrive at `CommRing ℝ`, we say a *diamond* has occurred (visually, two paths starting and ending in same nodes enclose a shape of diamond). Whenever diamond occurs, the elaborator will try to unify these instances and see if they are definitionally equal; if yes, fine; if not, then there will be an error cast “fails to synthesise an instance for...”.

Diamond issue is one of those “tiny but huge” problems during the formalisation, sometimes it might even be a major bottleneck. It forces us to think very carefully what structures are to be carried by the type, carefully design the hypothesis fields as well as the source instance declarations.

Here is an example. Our initial definition of `frobeniusInnerProduct` followed by specialising the following generic inner product instance to $M = 1$:

Inner Product Induced by Positive semi-definite Matrix

[View Source](#)

```
332 /-- A positive semi-definite matrix `M` induces an inner product on `Matrix n n ℚ` :
333 `

```

Mathematically this is OK, but soon we encountered an unavoidable diamond issue.

We want the underlying norm²⁰ to satisfy the normed ring bound $\|AB\| \leq \|A\|\|B\|$, namely, we should be able to infer a `NormedRing (Matrix n n ℚ)` instance. But here the underlying norm is the square root of trace re `(x * M * x†).trace` which does not induce a generic `NormedRing` instance. The Frobenius norm (the L_2 norm on matrices considered as an n^2 -dimensional space), on the other hand, has the required instance `Matrix.frobeniusNormedRing`, so we tried to utilise it.

However, Frobenius norm is not definitionally equal with the trace norm specialised to $M = 1$. This means, if we insist (1) using `toMatrixInnerProductSpace` as our inner product space and (2) using Frobenius norm with `frobeniusNormedRing` to infer the normed-ring instance, then there will always be a diamond issue at the norm level — that `toMatrixInnerProductSpace.toSeminormedAddCommGroup` is unable to be unified with `frobeniusSeminormedAddCommGroup`.

Therefore, we have to abandon either (1) or (2). If abandoning (1), then the only mathematical gap is to prove that the square of the Frobenius norm equal to re `(x * x†).trace`. If abandoning (2), then we need to prove that square root of re `(x * 1 * x†).trace` satisfies the normed-ring bound, but the easiest way is to show that this equals to the Frobenius norm and then explicitly call the instance — which is the same mathematical gap anyway with even more steps and simplifications needed. Hence, we chose to keep (1) as our final approach to `frobeniusInner/ProductSpace` declarations.

Lean and Mathlib have put a considerable software engineering efforts to mitigate the potential diamonds created by its declarations. Here we mention some practices that also made appearance in our project.

2.4.1 Scope Control by `scoped` and `local`

Albeit we say we want to *infer only one* instance, this does not mean we should *declare only one* instance. The set of n by n matrices can carry many different norm and inner product structures, each been used in different contexts; they all have to be declared. All we need is to prevent these different instances appearing

²⁰In Mathlib, `InnerProductSpace` extends `SeminormedAddCommGroup`, which means each inner product has an underlying norm.

in one context and being able to tell Lean “for this file/section, use ... as the official instance”. The ultimate weapon is again scope control.

Our practice of instance scoping for the “Frobenius family” follows Mathlib/Analysis/Matrix/Normed. It has declared three families of norm instances and each is scoped in a different namespace — including the Frobenius family scoped under the namespace `Matrix.Norms.Frobenius`. Moreover, they call these “non-instances” which is reflected in the following common practice of Mathlib:

- These “non-instances”, a.k.a. multiple viable structures of same class, are declared using raw `def` plus the attribute `scoped instance`. We extend this convention to our Frobenius inner product declarations; and whenever we want such instance, do `open scoped`.
- The `instance` declaration, or equivalently `def` tagged with `@[instance]`, is used only to declare “for sure” canonical instances. This include generic ones like `Ring.toNonUnitalRing`, and specific unanimous ones like `Ring R`. They are usually not scoped in namespace (though can be prefixed by).

Another common keyword is “`local`”. It tells Lean: “activate `decl_name` only within the current file / section”. Unlike `scoped` instances, `local` instances are used *only to help instance synthesis happening in the current file / section*, outside the file / section they can no longer be activated. For example, we write the following lines to just support the synthesis of the later-stage Frobenius inner product declarations:

```
attribute [local instance] Matrix.frobeniusSeminormedAddCommGroup
attribute [local instance] Matrix.frobeniusNormedSpace
```

We also tagged `local instance` to our inner product declarations 2.6. Here is our considerations:

- We want their names to still live in the namespace `Matrix`, highlighting that they are specialised to the `Matrix` type. But whenever we open `Matrix` for other reasons we do not want to activate them in instance synthesis. So we tagged them `local`.
- In this way, we can still explicitly call them with prefix `Matrix`. But if we do want to activate them as instances implicit in the environment, we are allowed to do this:

Scoping Instances Under External Namespaces
 View Source

```

69 namespace Norms.Frobenius
70
71 attribute [scoped instance]
72     Matrix.frobeniusInner
73     Matrix.frobeniusInnerProductSpace

```

Now we can activate them using `scoped Norms.Frobenius`. Similarly, we put them as `scoped` instances in namespace `FrobeniusMat`, so that `open scoped FrobeniusMat` can activate them and other instances altogether that are needed to specialise the `LiebConcavity` to matrix versions.

The `local` instances can also help to improve runtime efficiency, as the elaborator will prioritise them during instance search. We will showcase one such practice in section 3.3.3.

2.4.2 Transparency Control

We have been talking about “elaborator trying to unfold definitions and reduce expressions...” all way long. Lean has a special parameter — *transparency*, or *reducibility* — that controls to what extend the content of one definition can be seen/unfolded during elaboration. A common transparency level assumed by some

declarations is “semireducible”, which tells the elaborator to not automatically reduce it during *implicit checks for definitional equality*, but it will be reduced when we use `rfl`. More practical information can be found in “Library Style Guidelines” by Lean community.

But if at the end of the day all we want is to check if two things are definitionally equal, especially during instance synthesis, why not set everything to be completely reducible so that we will have best shot for the unification but bother allow varying degrees of transparency?

Our main character is the option “`backward.isDefEq.respectTransparency`”, which handles how implicit arguments — in particular those that involve declarations at semireducible transparency — are handled in checks for definitional equality during automation. If it is set to `true`, then the semireducible declarations won’t be automatically unfolded during checks for definitional equality (to “respect the transparency”), otherwise they will be reduced. Here is the reason that the default setting is now `true`:

Why `true` is now the default: The transparency bump meant that every speculative `isDefEq` call in proof automation could trigger expensive unfolding of semireducible definitions on implicit arguments — and most of these calls fail. This eventually became a performance bottleneck in Mathlib (doc-string of `backward.isDefEq.respectTransparency`).

In other words, experiences have found that many definitional unfolding are unnecessary because they are called at a wrong path for instance search.

However, this default cannot take care of the cases where unfolding of semireducible declarations *is* necessary. This has been one main troubles encountered in our project — having unexpected synthesis failures which we initially thought were the result of insufficient type class hypotheses. Examples:

- In all the matrix versions of the Lieb’s theorems, we cannot write $A \wedge q$, real powers of matrices, because the `HPow` instance is not synthesised.
- In the proof of Operator Jensen’s Inequality (the `apply` example), we cannot apply `ConvexOn.map_sum_le`.
- In proving that diagonal matrix over a `CStarAlgebra` is non-negative iff all its diagonal entries are non-negative, one key supporting result `nonneg_iff_sa_spectrum_nonneg` cannot be used.

Putting the line `set_option backward.isDefEq.respectTransparency false` in front of the declaration resolve all these issues. After knowing that transparency issue could be common in formalisation tasks that involve heavy type class assumptions like continuous functional calculus and `CStarAlgebra`, we learned to (1) always set this option to be `false` when starting a file, (2) after all works done, disable it to see which places actually need it, and (3) put a local option change in front of these places.

3 Action: From Matrix to HStarAlgebra

I will describe one of the most important decisions made in the project: the definition of a new type class `HStarAlgebra`. I will talk about how this decision is arrived from the mathematical point of view, how it, its accompanied API's are formalised, and describe some difficulties encountered, seeing theories in practice.

3.1 Mathematics

The original statement of Lieb's Concavity Theorem [NEE13; ENE11] takes place in the space of n by n complex matrices $M_n(\mathbb{C})$ endowed with Frobenius inner product defined by

$$\langle X, Y \rangle := \text{Tr}(Y^\dagger X).$$

The proof in [NEE13, Theorem 1.2] depends on the result of joint operator convexity of generalised perspective functions [ENE11, Theorem 2.5], which is also defined upon the matrix context. It is then specialised to a joint operator convexity of power functions acting via continuous functional calculus²¹ on Hermitian linear operators (left and right multiplications on $M_n(\mathbb{C})$ by Hermitian matrices), producing essentially an convex inequality at the level of linear operators and is then pushed to a convex inequality on inner product on $M_n(\mathbb{C})$, concluding the proof.

We realised that we have not really used anything special about $M_n(\mathbb{C})$ in this proof. This is true even back in the context of perspective function: roughly while the proof of joint operator convexity relies on Operator Jensen's Inequality [HP82], the finite-dimensional hypothesis on the operator space can be dropped and replaced with any structure that admits continuous functional calculus (so that applying real-valued function make sense) and with nice order structure.

Remark 3.1. This also means we need to define “operator convex/concave”, prove Operator Jensen's Inequality and the joint operator convexity of perspective functions in more general contexts. We did it: we formalised all of them in the context of unital C^* -algebra; we adopted the C^* -version of Operator Jensen's Inequality from [LW14, Theorem 2.2] and we realised that the argument in [ENE11] generalises straightforwardly to (ordered) unital C^* -algebras.

Here are some pre-thoughts.

- (1) For the underlying space, at the bare minimal, we need only an inner product space H over field k that also has a multiplication defined so that left, right multiplication make sense. For compatibility of multiplication and the scalar field action we then require H to be a k -algebra.
- (2) It also needs to have a star operation (star $\star = \dagger$ conjugate transpose for complex matrices). Speaking of the star structure, we furtherly require that H admits continuous functional calculus on its self-adjoint elements (Hermitian matrices), plus a (star) order structure so that taking real powers for non-negative / strictly positive elements (positive semi-definite / positive definite matrices) make sense.
- (3) For the space $B(H)$ of bounded linear operators on H , the order and star algebra structures plus continuous functional calculus must also show up so that we can safely talk about the composition of linear operators with real-valued functions applied as required in the definition of generalised perspective functions. The star operation unsurprisingly should be the (Hermitian) adjoint operation characterised in terms of inner product, and the order should be equivalent to the notion of a “positive operator”:

$$0 \leq \Phi \in B(H) \iff 0 \leq \text{Re}\langle \Phi X, X \rangle = \text{Re}\langle X, \Phi X \rangle \quad \text{for all } X \in H.$$

²¹Literatures [ENE11; Bha97] more commonly defined operator convexity/concavity via handy-waving in spectral decomposition / diagonalisation rather than using continuous functional calculus. While straightforward to say in words, this approach is hardly generalisable and suffers clean, well-defined formulae for the purpose of formalisation.

Roughly this is the level of generality we should work on. But we cannot ignore the fact that, in the matrix setup the inner product is defined using the trace function which affords it some special properties. An immediate one, in the presence of star-algebra structure, is induced by the cyclicity of trace map:

$$\langle AX, Y \rangle = \text{Tr}(Y(AX)^*) = \text{Tr}(YX^*A^*) = \text{Tr}(A^*YX^*) = \langle X, A^*Y \rangle. \quad (3.1)$$

Equation 3.1 is equivalent to the nice property that the adjoint of the left multiplication operator $L_A : X \mapsto AX$ is L_{A^*} , in other words star commutes with $L : A \mapsto L_A$. Similarly,

$$\langle XA, Y \rangle = \text{Tr}(Y(XA)^*) = \text{Tr}(YA^*X^*) = \langle X, YA^* \rangle, \quad (3.2)$$

i.e. right multiplication $R : A \mapsto (R_A : X \mapsto XA)$ commutes with star too.

These two equations look like desirable extra properties to have in our setting, but we still ask (1) if we can obtain sensible weaker assumptions than these and (2) if we need to add other assumptions. It turns out that they are *necessary and (roughly) sufficient*.

Here is the justification for necessity. One seemingly trivial thing we need to show is that, for $r \in \mathbb{R}$ and $A \in H$, whenever taking the power $(L_A)^r$ (likewise for R_A) makes sense, we must have $(L_A)^r(X) = A^r X$ (likewise $(R_A)^r = XA^r$). In other words, $(L_A)^r = L_{A^r}$ (likewise $(R_A)^r = R_{A^r}$). This fact has been implicitly assumed in the final simplification step of proof of Lieb that applies the powers of left/right multiplication operators. The “Mathlib way” of showing them in Lean is to show that L and R , as operators $H \rightarrow B(H)$, commutes with continuous functional calculus (we write “`cfc f`” in Lean for continuous functional calculus with f):

$$\text{cfc } f(L_A) = L_{\text{cfc } f(A)} \quad \text{and} \quad \text{cfc } f(R_A) = R_{\text{cfc } f(A)}, \quad (3.3)$$

and then specialises to f being real powers.

Luckily, Mathlib provides us the necessary bridge `StarHomClass.map_cfc`, so that when L and R are continuous and that they are star-algebra homomorphisms, we got the Equation 3.3. Being a star-algebra homomorphism in particular requires commuting with star, but L and R commuting with star is equivalent to Equations 3.1 and 3.2, QED.

These equations are also sufficient, provided also that multiplication in H is a “uniformly bounded” operation — more specifically we can assume (by scaling if necessary) that H is a normed ring, namely the norm $\|\cdot\|$ satisfies $\|AB\| \leq \|A\|\|B\|$ for all $A, B \in H$. With this assumption, we can show both L and R are continuous map from H to $B(H)$. It is easy to check that they are algebra homomorphisms (except the right multiplication is strictly speaking a homomorphism into opposite ring of $B(H)$), and they can then be upgraded to a star-algebra hom using these equations. We add these to the list:

- (4) H is a normed ring.
- (5) Inner product of H must satisfy Equations 3.1 and 3.2.

There is yet one last subtlety to consider: that left and right multiplication preserves non-negativity, i.e. if $0 \leq A$ in H then $0 \leq L_A$ and $0 \leq R_A$ in $B(H)$. This is relevant because Lieb’s Concavity assumes non-negativity only at the level of base space H , but this must be transported to the operator level $B(H)$ as that is what actually used by the argument of power means. Fortunately, this can also be readily resolved by Equations 3.1 and 3.2 and equivalent characterisations of non-negativity. In any star-ring H , there is a canonical notion of partial order $\leq$ defined by

$$0 \leq p \iff p \in \{ \text{additive closure of elements of the form } a^*a \}.$$

Star-rings with such property is called a `StarOrderedRing` in Mathlib. In many good cases, including all C^* -algebras, the above characterisation takes a simpler form $0 \leq p \iff p = a^*a$ for some a , and this

characterisation is equivalent, in our main targeted space $M_n(\mathbb{C})$, to the Loewner order induced by positive semi-definiteness (recall that a matrix is positive semi-definite iff it can be written as a product $A^\dagger A$). Our goal can then be proved by an induction principle on additive closure membership, with base case being direct consequence of Equations 3.1 and 3.2:

$$\operatorname{Re}\langle L_{A^*A}X, X \rangle = \operatorname{Re}\langle A^*AX, X \rangle = \operatorname{Re}\langle AX, AX \rangle \geq 0,$$

and vice versa for other cases.

It was a coincidence that we then discovered [Amb45] has already explored a structure almost identical to ours and the name H^* -algebra was used therein (except they do not assume H to have a unit), so we acknowledged it by using the same name.

3.2 Main Declarations and Results

Here is our `HStarAlgebra` class:

HStarAlgebra

[View Source](#)

```
54 variable (k : Type*) [RCLike k]
55
56 class HStarAlgebra (H : Type*) extends
57   NormedRing H, InnerProductSpace k H, Algebra k H, StarRing H where
58   protected inner_mul_left : ∀ (a x y : H), inner (a * x) y = inner x (star a * y)
59   protected inner_mul_right : ∀ (a x y : H), inner (x * a) y = inner x (y * star a)
```

A mere `HStarAlgebra` only has minimally-needed analytic and algebraic structures assumed. We did not put the continuous functional calculus and order structure on the definition because they belong to an external layer of interface, and they are added to the file only when relevant lemmas need them.

Remark 3.2. The type class `RCLike k` provides a unified interface for common operations we do on both $\mathbb{R}$ and $\mathbb{C}$. It is the main way in Mathlib to declare a field that can later be specialised to either the reals $\mathbb{R}$ and the complexes $\mathbb{C}$, and is mainly used when inner product structure present.

Immediately after the class definition we declared the following “useless thing”:

Export and Redeclared Axioms

[View Source](#)

```
61 @[simp]
62 theorem inner_left_mul_eq (a x y : H) :
63   ⟨a * x, y⟩ = ⟨x, star a * y⟩ :=
64   HStarAlgebra.inner_mul_left a x y
65
66 @[simp]
67 theorem inner_right_mul_eq (a x y : H) :
68   ⟨x, a * y⟩ = ⟨star a * x, y⟩ := by
69   rw [inner_left_mul_eq, star_star]
```

Although `inner_right_mul_eq` is new, `inner_left_mul_eq` simply repeats the axiom `inner_mul_left`. The reasons are two.

- The axioms `inner_mul_left`, `inner_mul_right` cannot be tagged by `@[simp]` to be automatically used by the family of `simp` tactics. So we need external versions to be tagged.

- More importantly this practice reflects the standard practice of abstraction principle in software engineering. It helps mitigate the issue that, if we modify an existing definition with data fields changes, then any downstream project that directly uses part of the old definition won't compile with this newer version imported. The downstream declarations should always try to use the associated external APIs whose utilities are unaffected by the black box changes in the definitions.

For this reason we also tagged the two axioms by the keyword “**protected**” in the definition field, which has the effect that if they were to be called then the prefix `HStarAlgebra.` must be typed — to discourage the users, for real.

3.2.1 Algebra Homomorphism and some Properties

Next, we bundled the left multiplication as an algebra homomorphism from H to $B(H)$:

Lmul as an Algebra Hom

 [View Source](#)

```

86  /-- Left multiplication as an algebra homomorphism `H →a [k] (H →L [k] H)` . -/
87  def LmulAlgHom : H →a [k] (H →L [k] H) where
88    toFun a :=
89      { toLinearMap := LinearMap.mulLeft k a
90        cont       := continuous_const_mul a }
91    map_one' := by ext; simp
92    map_mul' := fun _ _ => by ext; simp
93    map_zero' := by ext; simp
94    map_add' := fun _ _ => by ext; simp [add_mul]
95    commutes' := fun _ => by ext; simp [Algebra.algebraMap_eq_smul_one]
96
97  /-- Left multiplication by `a`, as a continuous linear map `H →L [k] H` . -/
98  def Lmul (a : H) : H →L [k] H := LmulAlgHom k a

```

The bundled `LmulAlgHom`, the “ L ” map, is used mainly in the main theorems whose proof requires the bundled algebra-hom structure. But in downstream applications that calls “ L_A ” for some $A \in H$, we use the abbreviated `Lmul k a` instead; to assist applications, it is also followed by a sequence of basic `simp` lemmas, like `Lmul_zero` which says `Lmul k 0 = (0 : H →L [k] H)`.

Next we bundled the right multiplication as an algebra hom into $B(H)^{\text{op}}$:

Rmul as an Hom into Opposite Algebra

 [View Source](#)

```

137  /-- Right multiplication is a ring *anti*-homomorphism on `H` (`R_{ab} = R_b ∘ R_a`).
138  Thus it is bundled as an algebra homomorphism `H →a [k] (H →L [k] H)mop` . -/
139  def RmulAlgHom : H →a [k] (H →L [k] H)mop where
140    toFun a := op
141    { toLinearMap := LinearMap.mulRight k a
142      cont       := continuous_mul_const a }
143    map_one' := by apply congrArg op; ext; simp
144    ...
145  /-- Right multiplication by `a`, as a continuous linear map `H →L [k] H` . -/
146  def Rmul (a : H) : H →L [k] H := unop (RmulAlgHom k a)

```

Here “`mop`” stands for “multiplicative opposite structure”. Unlike left multiplication, the abbreviation `Rmul k a` and `RmulAlgHom k a` are of different types. For this reason many theorems about right multiplication are formalised in a pair: one uses the `RmulAlgHom` whose proof is structurally committed to the type and lives in the opposite algebra, another uses the abbrev `Rmul` whose proof involves “`unop`” the `RmulAlgHom`

version and lives in the usual operator algebra. There is one exception that “unop” won’t work due to lack of Mathlib declarations, which we will soon discuss below in the CFC section.

We then documented the basic properties of left and right multiplications: continuity, unit-preserving (if A is invertible then so are L_A and R_A), symmetry (when A is self-adjoint then $\langle L_A X, Y \rangle = \langle X, L_A Y \rangle$, etc.), and non-negativity preserving. We showcase the non-negativity preserving here.

Left Multiplication Preserves Non-negativity

[View Source](#)

```

258 lemma re_inner_Lmul_star_mul_self_nonneg (s x : H) :
259   0 ≤ RCLike.re «(Lmul k (star s * s)) x, x» := by
260   simp_rw [Lmul_apply, mul_assoc, <- inner_right_mul_eq]
261   exact inner_self_nonneg
262
263 variable [PartialOrder H] [StarOrderedRing H]
264
265 theorem Lmul_isPositive {a : H} (ha : 0 ≤ a) : (Lmul k a).IsPositive := by
266   refine ⟨Lmul_isSymmetric k (IsSelfAdjoint.of_nonneg ha), fun x => ?_⟩
267   simp only [ContinuousLinearMap.reApplyInnerSelf_apply]
268   rw [StarOrderedRing.nonneg_iff] at ha
269   induction ha using AddSubmonoid.closure_induction with
270   | mem b hb =>
271     obtain ⟨s, rfl⟩ := hb
272     exact re_inner_Lmul_star_mul_self_nonneg k s x
273   | zero => simp
274   | add b c _ _ ihb ihc =>
275     rw [Lmul_add, ContinuousLinearMap.add_apply, inner_add_left, map_add RCLike.re]
276     exact add_nonneg ihb ihc
277
278 theorem Lmul_nonneg {a : H} (ha : 0 ≤ a) : 0 ≤ Lmul k a :=
279   (Lmul _ a).nonneg_iff_isPositive.mpr (Lmul_isPositive _ ha)

```

Noteworthy is the proof of `Lmul_isPositive` that uses an induction principle for closure membership (the set of star-positive elements form an additive sub-monoid), whose “base case” is the lemma `re_inner_Lmul_star_mul_self_nonneg`. This again is a principle derived from the recursor. Finally, it is transformed to the $0 \leq \text{Lmul } k \ a$ version that suits our later application that confers to the `StarOrderedRing` structure. The versions for `Rmul` are completely analogous.

All above results only depend on algebra homomorphism structure (even the linearity is enough). In next stage, we finally upgrade L and R as star-algebra homomorphisms.

3.2.2 Star-Algebra Homomorphisms and CFC/Rpow Commutativity

We want the star operation on $B(H)$ being the adjoint, not any star. By virtue of Mathlib, as long as we put H to be a complete space, this can be automatically instantiated:

Rmul as a StarAlgHom

[View Source](#)

```

326 -- notes: with H complete, star/adjoint structure can be instantiated on `H → L[k] H`.
327 variable [CompleteSpace H]
328
329 theorem Rmul_star (a : H) :
330   Rmul k (star a) = star (Rmul k a) := by
331   rw [ContinuousLinearMap.star_eq_adjoint]
332   exact (ContinuousLinearMap.eq_adjoint_iff _ _).mpr fun _ => by
333     simp only [Rmul_apply, inner_mul_left_eq, star_star]

```

Continued on next page...

```

334
335 /-- Right multiplication as a star algebra homomorphism. -/
336 def RmulStarAlgHom : H →*a[k] (H →L[k] H)mop :=
337   { RmulAlgHom k with
338     map_star' := fun a => congrArg op (Rmul_star k a) }

```

We can see that the merit of the bundled approach is that an existing bundled one (`AlgHom`) can be lifted to another one (`StarAlgHom`) just by filling out the missing fields (`map_star'`). Unfortunately since this is a new type we have to introduce a new name, but this is not that bad, at least we can then tell from the name explicitly what kind of structure we are dealing with. We also put the proof of `map_star'` outside not just for modularity but to get an independently-usable lemma — necessary for right multiplication because `Rmul_star` works in $H \rightarrow_L[k] H$ but `map_star'` works in $(H \rightarrow_L[k] H)^{\text{mop}}$.

Now we are ready for the central results: L and R commute with continuous functional calculus. We put them in a new `section` `CFC` and starts with the following variable blocks:

```

variable [CompleteSpace H] [Algebra R H] [IsScalarTower R k H]
variable [ContinuousFunctionalCalculus R H IsSelfAdjoint]

```

We remark that `HStarAlgebra k H` alone does not imply the `Algebra R H` instance and we also need to use `IsScalarTower R k H` to dictate the behaviour of $\mathbb{R}$ -action. The type class `ContinuousFunctionalCalculus` has some interesting design decisions, the interested reader may refer to [DL25]. For our purpose, we only need to know that declaring `[ContinuousFunctionalCalculus R H IsSelfAdjoint]` allows us to, for real functions $f : \mathbb{R} \rightarrow \mathbb{R}$, write `cfc f : H → H` and apply it meaningfully to self-adjoint elements.

Remark 3.3. In Lean, functions are usually defined at the global level: if we have a function that is mathematically undefined on some point (like $1/x$), then we assign some *trash value* (usually 0) to that point to complete the domain. The benefit is that we don't need to build new function type with a restricted domain. In our case, if $x : H$ is not self-adjoint, then `(cfc f) x` has trash value 0.

Here is part of the main declarations for left multiplication (we omit the proof for logical clarity):

Left Multiplication Commutes with CFC/Rpow

[View Source](#)

```

394 /-- Left multiplication commutes with the continuous functional calculus:
395   `L_{f(a)} = f(L_a)` for self-adjoint `a` and continuous `f`. -/
396 theorem Lmul_map_cfc (f : R → R) (a : H)
397   (hf : ContinuousOn f (spectrum R a) := by cfc_cont_tac)
398   (ha : IsSelfAdjoint a := by cfc_tac) :
399   LmulStarAlgHom k (cfc f a) = cfc f (LmulStarAlgHom k a)
400
401 variable [PartialOrder H] [StarOrderedRing H]
402
403 /-- Left multiplication commutes with nonneg real powers: `L_a ^ r = L_{a ^ r}`. -/
404 theorem Lmul_rpow_nonneg' {r : R} {a : H} (hr : 0 ≤ r) (ha : 0 ≤ a := by cfc_tac) :
405   cfc (fun x : R ↦ x ^ r) (Lmul k a) = Lmul k (cfc (fun x : R ↦ x ^ r) a)
406
407 theorem Lmul_rpow_nonneg_apply' {r : R} {a : H} (x : H)
408   (hr : 0 ≤ r) (ha : 0 ≤ a := by cfc_tac) :
409   (cfc (fun x : R ↦ x ^ r) (Lmul k a)) x = cfc (fun x : R ↦ x ^ r) a * x
410
411 variable [NonnegSpectrumClass R H]
412 variable [StarOrderedRing (H →L[k] H)] [NonnegSpectrumClass R (H →L[k] H)]
413

```

Continued on next page...

```

414 /-- `L_a ^ r` acts on `x` is `a ^ r * x` for nonnegative power. -/
415 theorem Lmul_rpow_nonneg_apply
416   {r : ℝ} {a : H} (x : H) (hr : 0 ≤ r) (ha : 0 ≤ a := by cfc_tac) :
417     ((Lmul k a) ^ r) x = (a ^ r) * x

```

`Lmul_map_cfc` is the source of truth. It is deduced from the Mathlib declaration `StarAlgHom.map_cfc`. The rest theorems are, sequentially, corollaries of former and are proved by straightforward `rw` and `exact`.

We also have distinguished two ways to express “taking real powers”: one is to use the raw `cfc` applying to real power functions as in the middle two primed theorems, and another is the direct “`^`” symbol used in the unprimed theorem `Lmul_rpow_nonneg_apply`. The raw `cfc` version requires only `PartialOrder` and `StarOrderedRing` instance. The `^` symbol — which refers to the polymorphic type `Pow` (or `HPow`) of power operation — in addition needs `NonnegSpectrumClass` (asserts non-negative elements are precisely those whose spectrum is non-negative), see the Source. Hence, they are declarations living in different levels.

Here is the right multiplication counterpart (with assumptions omitted):

Right Multiplication Commutes with CFC/Rpow

[View Source](#)

```

454 theorem Rmul_map_cfc :
455   RmulStarAlgHom k (cfc f a) = cfc f (RmulStarAlgHom k a)
456
457 theorem Rmul_rpow_nonneg_op :
458   cfc (fun x : ℝ ↦ x ^ r) (RmulStarAlgHom k a)
459   = RmulStarAlgHom k (cfc (fun x : ℝ ↦ x ^ r) a)
460
461 theorem Rmul_rpow_nonneg' :
462   cfc (fun x : ℝ ↦ x ^ r) (Rmul k a) = Rmul k (cfc (fun x : ℝ ↦ x ^ r) a)
463
464 theorem Rmul_rpow_nonneg_apply' :
465   (cfc (fun x : ℝ ↦ x ^ r) (Rmul k a)) x = x * cfc (fun x : ℝ ↦ x ^ r) a
466
467 theorem Rmul_rpow_nonneg_apply :
468   ((Rmul k a) ^ r) x = x * a ^ r

```

Comparing to the `Lmul` counterpart, it has an extra intermediate step `Rmul_rpow_nonneg_op` with real powers taking in the opposite algebra $(H \rightarrow L[k] H)^{\text{mop}}$. This is because the source of truth `Rmul_map_cfc` has to be declared with the opposite algebra, so we can only immediately specialise to such a version.

We then try to “unop” it to get the clean `Rmul` version that lives in $(H \rightarrow L[k] H)$. However, we encounter the only case in this file where direct unop is not yet supported by Mathlib. To fill in the gap, we created a separate file, “OpStar”, that used the same trick of (1) bundling the map $x \mapsto \text{op}(\text{star } x)$ as a star-algebra homomorphism (equivalence) and (2) proves that it commutes with CFC and Rpow with suitable specialised versions that appeal to our unop demands. With this gap filled, the rest are easy corollaries.

Besides the `_nonneg_` versions, we also have `_StrictlyPositive_` versions which has no assumption on the power `r` but requires `a` to be `IsStrictlyPositive` — non-negative and invertible.²² For matrices the property `IsStrictlyPositive` corresponds to being positive semi-definite, to whom taking negative powers is allowed.

²²This is the existing definition in Mathlib, but is currently only supported on *unital* algebra. A more general definition that takes care the non-unital case is on the way.

3.3 Obstacles

Actually, there had been several version changes leading to the present definition of `HStarAlgebra` — all have to do with the typeclass synthesis troubles prompted alone by its definition.

3.3.1 Puzzle with `extends` and Diamond Digging

Here's the initial version:

```
class HStarAlgebra (k : Type*) (H : Type*) [RCLike k]
  [SeminormedAddCommGroup H] extends
    Semiring H, StarRing H, InnerProductSpace k H where
  ...
```

We had `SeminormedAddCommGroup H` assumed outside the `extends` chain, which was meant to merely replicate the exact assumption fields of `InnerProductSpace k H`. We also did not put `Algebra` inside.

But when we were defining `LmulAlgHom`, the algebra hom class `H →a [k] (H →L [k] H)` did not synthesise and the error message was: “failed to synthesise `Module k H`”. This was very strange, because theoretically the k -linear structure should be inferable from the k -inner product structure. We tried to left it as it was and manually put a `Module k H` variable block in front of `LmulAlgHom`, to double check if it was simply that Lean failed to find the `Module` instance or other issues. Then several new errors prompted, and we added more failed-to-synthesised type classes in the block, until:

```
variable [Module k H] [Semiring (H →L [k] H)] [Algebra k H] [Algebra k (H →L [k] H)]
```

Now `H →a [k] (H →L [k] H)` was synthesised.

However, the term `LinearMap.mulLeft k a` in the proof field of `toFun` did not synthesise and the error prompted “failed to synthesise `SMulCommClass k H H`”. This was a very low level class, so we realised that the root cause of this series of errors was likely a diamond, e.g. having two definitionally equal instances of scalar multiplication or something. We first decided, as `Algebra` class was ultimately needed by the algebra homomorphism, to put it inside the definition.

```
class HStarAlgebra (k : Type*) (H : Type*) [RCLike k] extends
  [SeminormedAddCommGroup H] extends
    Semiring H, StarRing H, Algebra k H, InnerProductSpace k H where
  ...
```

Now we did not have `Module k H` synthesis failure. But `[Semiring (H →L [k] H)]` still cannot be synthesised... After experimentations, we discovered that after putting `SeminormedAddCommGroup H` inside `extends` chain, all old errors disappeared!

Below we will show how to dig into the root cause of which using Lean’s diagnosis tool, in particular we will figure out what made `Semiring (H →L [k] H)` failed to synthesise before.

1. First we need to check the instance declaration of `Semiring (H →L [k] H)` in the Mathlib source file, to see what type classes are assumed by it so that we may narrow our investigation to these assumed ones. We identified `ContinuousLinearMap.semiring`:

```
587 instance semiring [ContinuousAdd M1] : Semiring (M1 → L[R1] M1) where ...
```

So the critical class is `ContinuousAdd`. This class asserts that the binary addition operation is a continuous map — indeed, mathematically we need it to ensure that addition of two continuous maps to be continuous so that we at least can have an addition operation well-defined in the continuous linear operator algebra.

- Let's see if the error was propagated from this `ContinuousAdd` instance. Put `SeminormedAddCommGroup` inside the `extends` chain, and let us check the synthesis path.

```
set_option trace.Meta.synthInstance true in
example : ContinuousAdd H := inferInstance
```

The “`inferInstance`” tells Lean to automatically infer and synthesise the instance. Then in the **infoview** we look at the entry `[Meta.synthInstance] ContinuousAdd H`, opening its sub-entries and scroll all the way down to see the path that finds it:

```
SeminormedAddCommGroup.toIsTopologicalAddGroup.toContinuousAdd
```

This path confirms that: `SeminormedAddCommGroup` alone can reach `ContinuousAdd`.

- Next, we re-allocate `SeminormedAddCommGroup` outside the `extends` chain. Immediately, `inferInstance` does not succeed in inferring `ContinuousAdd`. This confirms that (partly) the synthesis failure was propagated from it. How about use `SeminormedAddCommGroup.toIsTopologicalAddGroup.toContinuousAdd` directly to infer the instance?

After doing so, we see the critical error message:

```
/- Type mismatch
   SeminormedAddCommGroup.toIsTopologicalAddGroup.toContinuousAdd
has type -/
@ContinuousAdd ?m.7 PseudoMetricSpace.toUniformSpace.toTopologicalSpace
   SeminormedAddCommGroup.toSeminormedAddGroup.toAdd
/- but is expected to have type -/
@ContinuousAdd H PseudoMetricSpace.toUniformSpace.toTopologicalSpace
   (inst†.toSemiring k).toNonAssocSemiring.toDistrib.toAdd
```

The last two lines of these two mismatched types are the different paths to the `Add H` instances (i.e. the addition operation) with origins `SeminormedAddCommGroup` and `Semiring` respectively. This confirms that we have a diamond issue: *the outer `SeminormedAddCommGroup` has introduced an extra, definitionally-unequal `Add` instance alongside the one from `Semiring`.*

When putting `SeminormedAddCommGroup H` and `Semiring H` inside the `extends` chain, their shared fields (including the `Add H`'s fields) are merged, so prompts the synthesis.

This concludes the diagnosis. The lesson is that we have to be very careful in the way we put structures together; putting things into `extends` chain can sometimes be a necessary step.

3.3.2 Completing the Hypothesis

The remaining issues have to do with insufficient typeclass hypotheses imposed, and the solution typically is to update the definition whenever the error asked. For example, when we tried to prove L and R are continuous functions we have to upgrade `SeminormedAddCommGroup` to `SeminormedRing H`, because the latter introduces the key property $\|AB\| \leq \|A\| \|B\|$ for all $A, B \in H$ that makes the continuity proof plausible by directly calling a Mathlib result:

Continuity of L

[View Source](#)

```
207  /-- The map `a ↦ L_a` is continuous w.r.t. the operator norm -/
208  theorem Lmul_continuous :
209    Continuous (LmulAlgHom k : H → H → L[k] H) :=
210    isBoundedBilinearMap_mul (k := k) (A := H) |>.toContinuousLinearMap.continuous
```

Again, we did not just add `SeminormedRing H` as an extra hypothesis for this theorem, because outside the `extends` chain it will cause the same diamond issue. Since `SeminormedRing` implies `Semiring` we can update our definition as:

```
class HStarAlgebra (k : Type*) (H : Type*) [RCLike k] extends
  SeminormedRing H, StarRing H, Algebra k H, InnerProductSpace k H where
  ...
```

We are one-step away from our final version which uses `NormedRing` instead of `SeminormedRing H`. What happened was that in the non-negativity lemmas, `IsPositive` requires `NormedAddCommGroup H` instance. `SeminormedRing` almost consumes it except a subtle aspect: `SeminormedRing` extends “`PseudoMetricSpace`” but `NormedAddCommGroup` extends `MetricSpace`, with the latter being strictly stronger than the former (which does not have the property that if two points have distance 0 then they are the same point). As a single `NormedRing` perfectly consumes both, we did this switch and arrived at the final version. This is the truth buried deep in history: we did not know `NormedRing H` should be a part of the definition (the analysis at mathematical section was hindsight), it was discovered gradually as we proceed in establishing more results.

3.3.3 Efficiency Concerns

Next, we mention a type class issue of different nature than previous ones: *slow unification*. We discovered that our CFC section — especially the right multiplication case — builds significantly slower than the rest part of the file. One first improvement we made was to make critical proof step explicit with `@`:

Proof of Left Multiplication Commutes with CFC

[View Source](#)

```
394  /-- Left multiplication commutes with the continuous functional calculus:
395  `L_{f(a)} = f(L_a)` for self-adjoint `a` and continuous `f`. -/
396  theorem Lmul_map_cfc (f : R → R) (a : H)
397    (hf : ContinuousOn f (spectrum R a) := by cfc_cont_tac)
398    (ha : IsSelfAdjoint a := by cfc_tac) :
399    LmulStarAlgHom k (cfc f a) = cfc f (LmulStarAlgHom k a) :=
400    (@LmulStarAlgHom k _ H _).map_cfc f a hf (@Lmul_continuous k _ H _) ha <|
401    @Lmul_isSelfAdjoint k _ H _ a ha
```

We discovered that just by making implicit H explicit the theorem builds significantly faster. However, the section as a whole still builds slow and the explication methods don't work quite well for other declarations.

The right multiplication section even took more than roughly twice times of the left multiplication counterpart to build. We need to look closely at what hinders the performance.

We follow the practices in the Leanprover community’s article “How to speed up a Mathlib file”:

1. Add the line `set_option profiler true` after the `imports` statements. The effect of which is that now in the infoview, for each declarations that took more than a threshold t ($t = 100\text{ms}$ by default) to build, we can see what actions in it took more than t milliseconds, which can be either typeclass inference, tactic execution, type checking, or elaboration.
2. Now we see almost all declarations within our CFC section, including the theorems statement and even some variable blocks involve slow steps in the building process. We identify three predominant factors:
 - “typeclass inference of `Algebra` took <not short time>”
 - “typeclass inference of `SMul` took <not short time>”
 - “typeclass inference of `Module` took <not short time>”
3. Experiment confirms that adding these `local instance` declarations²³ evidently boost the entire section’s performance:

```
local instance : SMul R (H → L[k] H) := ContinuousLinearMap.instSMul
local instance : Module R (H → L[k] H) := ContinuousLinearMap.module
local instance : Algebra R (H → L[k] H) := ContinuousLinearMap.algebra
```

i.e., it were the $\mathbb{R}$ -scalar/module/algebra instances on $H \rightarrow L[k] H$ that are the main blockers. This makes sense: we have only declared $\mathbb{R}$ -scalar action for the underlying space H , putting hard burden on elaborator to lift them to the operator algebra level.

4. We also discovered that the inference of `HPow` instances in those `..._rpow..._apply` theorem are slow, but the root cause should be the `Pow` instances (in this context, taking real powers via `cfc`). These instances were declared to have low priority in the Mathlib’s source file. But for our local purpose it is completely fine to have them instantiated with high priority:

```
local instance (priority := high) : Pow H R := CFC.instPowReal
local instance (priority := high) : Pow (H → L[k] H) R := CFC.instPowReal
```

5. The remaining slow steps have to do with tactics and type-checking. But we think it is unnecessary to make further changes, as we’ve been using very elementary tactic that does not involve heavy pattern-matching and, afterall, the whole section now builds with decent (though not perfect) speed.

Lean keeps an internal metric called *heartbeat* that counts “the number of ‘small’ memory allocations performed in a thread” [21.8 Timing, *The Lean Language Reference*]. In the case of theorem proving, both instance search and tactics application ask the system to allocate memory, so the number of heartbeat can be very informative in telling the time a piece of the code (can be a definition, a theorem, or an instance declaration) takes to build due to these actions. In practice, the worst case involves the programme halt with error message “maximum heartbeats N reached” (N is by default 200,000), which typically indicates

²³As suggested by the article, a better solution would be to find out what causes the type search to be slow. But this involves meticulous investigation of not just the current file but also relevant Mathlib source files. We kept it as a TODO.

that part of the proof ran into an infinite loop (e.g. `simp` rewrites between two goals back and forth), or the typeclass declarations are poorly designed so elaboration takes too much time.

In the Lean file, we can type “`set_option linter.countHeartbeats true`” to allow the infoview to display the heartbeat counts. We make a comparative study within the CFC section before and after implementing the above improvements:

Declaration	Heartbeats (before)	Heartbeats (after)
<code>Lmul_map_cfc</code>	4409	1139
<code>Lmul_rpow_nonneg_apply'</code>	4451	1184
<code>Lmul_rpow_nonneg_apply</code>	9426	3115
<code>Rmul_map_cfc</code>	5078	1820
<code>Rmul_rpow_nonneg_op</code>	5122	1865
<code>Rmul_rpow_nonneg_apply'</code>	13243	3635
<code>Rmul_rpow_nonneg_apply</code>	17476	4814

Table 2: Performance Boost by Local Instantiations

All declarations now take less than 1/3 of the original heartbeats. Note that there are visible growths of heartbeats when going from `_apply'` to `_apply` theorems for both left and right multiplications. Such growth in heartbeats can hardly be optimised, because the `_apply` results are established upon richer type classes like `NonnegSpectrumClass` and `StarOrderedRing`.

Conclusion and Outlook

The deeper lesson of this project lies in the more CS and engineering aspect. First is the principle of modularity, which involves defining appropriate abstraction layers that goes hand-in-hand with mathematical abstractions, building structural APIs, principled scoping of instance declarations. I have been enjoying building the `HStarAlgebra` declarations and the Frobenius family of instances, trying to follow the Mathlib standard. Second is the dirty part, which mostly involves understanding how elaborator works that affect the behaviours of the tactics and type class inferences. The automation works done by the elaborator — notably checking definitional equality and instance search — are fantastic intelligences without which the users will have too much proof burden, but we have to deal with the complications, to fix type-mismatches, trace the source of instance synthesis failures, and boost instance search efficiency.

This essay has deliberately de-emphasised the theorem-proving aspect of the formalisation because that aspect is less systematic and the user cases vary wildly for different tasks. It is my opinion that proof construction is in most cases a predictable activity of goal-chasing once we have very clear ideas what are the precise set of hypotheses and lemmas available to use, and have the mathematical complexities spread out into crystal clear sub-steps, the rest is experimentations with tactics and optimisations of code qualities. The more interesting aspect of theorem-proving is still the discovery of new abstraction levels the old theorems can live, and to be able to modularise the proof scheme into layers of mathematical contents that each can be independently developed and even expanded. Aside from formalising the main theorems like Lieb's Concavity, Operator Jensen's Inequality and joint convexity of perspective functions, this project has resulted in a series of natural candidates for upstreaming to Mathlib, ranging from lemmas in continuous functional calculus, classes of star-algebra homomorphisms, and new members to the Frobenius norm family, all live in the project's subdirectory `ForMathlib/`. They are in my TODOs for new PRs.

Since my understanding of formalisation and precision on the mathematical content having been growing along the course of the project, some designs, especially those made at the early-stages, admit rooms for improvements. There is even the possibility that the current `HStarAlgebra` class can be re-declared as / accompanied by a non-unital version and then to re-formalise Lieb's Concavity upon which. Again, this will be a future TODO.

References

- [Amb45] Warren Ambrose. “Structure theorems for a special class of Banach algebras”. In: *Trans. Amer. Math. Soc.* 57 (1945), pp. 364–386. DOI: 10.2307/1990182.
- [LeanThm] Jeremy Avigad et al. *Theorem Proving in Lean 4*. 2024. URL: https://lean-lang.org/theorem_proving_in_lean4/.
- [Bha97] Rajendra Bhatia. *Matrix Analysis*. Vol. 169. Graduate Texts in Mathematics. Springer-Verlag, New York, 1997. DOI: 10.1007/978-1-4612-0653-8.
- [DL25] Anatole Dedecker and Jireh Loreaux. *The continuous functional calculus in Lean*. 2025. arXiv: 2501.15639 [cs.LO].
- [ENE11] Ali Ebadian, Ismail Nikoufar, and Madjid Eshaghi Gordji. “Perspectives of matrix convex functions”. In: *Proc. Natl. Acad. Sci. USA* 108.18 (2011), pp. 7313–7314. DOI: 10.1073/pnas.1102518108.
- [HP82] Frank Hansen and Gert K. Pedersen. “Jensen’s inequality for operators and Löwner’s theorem”. In: *Math. Ann.* 258 (1982), pp. 229–241. DOI: 10.1007/BF01450679.
- [How80] William Alvin Howard. “The Formulae-as-Types Notion of Construction”. In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus, and Formalism*. New York: Academic Press, 1980, pp. 479–490.
- [LeanComm] Lean Community. URL: <https://leanprover-community.github.io/index.html>.
- [ei25] *Lean-LiebConcavity*. <https://github.com/eiKeViN/Lean-LiebConcavity>. Lean 4 formalization of Lieb’s concavity theorem. 2026.
- [LW14] Xiao Li and Wen Wu. “Operator Jensen’s inequality on C*-algebras”. In: *Acta Math. Sin. (Engl. Ser.)* 30.1 (2014), pp. 35–50. DOI: 10.1007/s10114-013-2065-8.
- [NEE13] Ismail Nikoufar, Ali Ebadian, and Madjid Eshaghi Gordji. “The simplest proof of Lieb concavity theorem”. In: *Adv. Math.* 248 (2013), pp. 531–533. DOI: 10.1016/j.aim.2013.07.019.
- [SU06] Morten Heine Sørensen and Paweł Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Elsevier, 2006. DOI: 10.1016/S0049-237X(06)80005-4.
- [LeanRef] The Lean FRO. *The Lean Language Reference*. URL: <https://lean-lang.org/doc/reference/latest/>.
- [Mathlib20] The Mathlib Community. “The Lean Mathematical Library”. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2020. ACM, 2020. DOI: 10.1145/3372885.3373824.